

Table of Contents

myAGV — ROS 1 vs ROS 2 Reference Guide

This guide explains **which packages, commands, and tools belong to ROS 1 and which belong to ROS 2** on the myAGV. The two versions are **not interchangeable** — a ROS 1 command will not work in a ROS 2 workspace and vice versa. Pick the version that matches your robot, then only use the column for that version.

The ROS software is already installed and built on myAGV. You do **not** need to clone or build anything for normal use — just run the commands below. Cloning and rebuilding are only needed if the installation becomes broken or unusable (see **Appendix A**).

How to tell which one you have Open a terminal on the AGV and run:

```
printenv ROS_DISTRO
```

- noetic → you are on **ROS 1**
- galactic → you are on **ROS 2**

1. At-a-glance comparison

Topic	ROS 1	ROS 2
Repository	myagv_ros	myagv_ros2
Jetson branch	myagv_ros_2023JN	galactic-JN
ROS distro	Melodic / Noetic	Galactic
Ubuntu	18.04 (Melodic) / 20.04 (Noetic)	20.04
Workspace (typical)	~/myagv_ros1	~/myagv_ros2
Launch command	roslaunch <pkg> <file>.launch	ros2 launch <pkg> <file>.launch.py
Run a node	roslaunch <pkg> <node>	ros2 run <pkg> <node>
Central process	roscore (required)	none (DDS, auto-discovery)
Navigation stack	move_base (Navigation)	Nav2 (navigation2)
Build tool (<i>rebuild only</i>)	catkin_make	colcon build

2. Package names — the important difference

Several packages exist in **both** versions but with **different names**. Using the wrong name is the most common mistake.

Function	ROS 1 package	ROS 2 package
Base driver / odometry	myagv_odometry	myagv_odometry <i>(same name, different code)</i>
Navigation	myagv_navigation	myagv_navigation2
Robot model (URDF)	myagv_urdf	myagv_description
Teleop (keyboard)	myagv_teleop	teleop_twist_keyboard
Teleop (PS2 gamepad)	myagv_ps2	<i>(not included)</i>
Lidar driver	ydliar_ros_driver	ydliar_ros2_driver
Camera	jetson_camera	<i>(uses vision_opencv)</i>
Odometry fusion (EKF)	robot_pose_ekf	robot_localization
SLAM (gmapping)	<i>(gmapping via navigation)</i>	slam_gmapping
Visual SLAM	ORB_SLAM2, rtabmap_ros	<i>(not included)</i>
Multi-goal RViz plugin	navi_multi_goals_pub_rviz_plugin	<i>(not included)</i>

⚠ **myagv_navigation (ROS 1) ≠ myagv_navigation2 (ROS 2)**. The “2” is not optional — it’s a different package.

3. Bringing the robot up

ROS 1

```
# ROS 1 needs a master running first (roslaunch starts one automatically)
roslaunch myagv_odometry myagv_active.launch    # base + odometry + Li
dar
```

ROS 2

```
# No roscore needed. Launch files end in .launch.py
ros2 launch myagv_odometry myagv_active.launch.py    # base + odometry
ros2 launch ydliar_ros2_driver ydliar_launch.py      # Lidar (separate
Launch)
```

4. SLAM (build a map)

ROS 1

```
# Uses gmapping through the navigation package
roslaunch myagv_navigation myagv_slam_laser.launch
# drive around, then save:
roslaunch map_server map_saver -f ~/map
```

ROS 2

```
ros2 launch slam_gmapping slam_gmapping.launch.py
# drive around, then save:
ros2 run nav2_map_server map_saver_cli -f ~/map
```

Note: map saving tool differs — ROS 1 = map_server map_saver, ROS 2 = nav2_map_server map_saver_cli.

5. Navigation (use a saved map)

ROS 1 — move_base

```
roslaunch myagv_navigation navigation_active.launch
# Send goals with the "2D Nav Goal" button in RViz.
```

ROS 2 — Nav2

```
ros2 launch myagv_navigation2 navigation2_active.launch.py
# Send goals with the "Nav2 Goal" button in RViz2.
```

6. Teleop (manual driving)

ROS 1

```
roslaunch myagv_teleop myagv_teleop.launch # keyboard
# PS2 gamepad also available via myagv_ps2
```

ROS 2

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

7. Everyday command cheat-sheet

What you want to
do

ROS 1

ROS 2

Start core	roscore	(not needed)
Launch a file	roslaunch pkg file.launch	ros2 launch pkg file.launch.py
Run a node	roslaunch pkg node	ros2 run pkg node
List topics	rostopic list	ros2 topic list

Echo a topic	<code>rostopic echo /odom</code>	<code>ros2 topic echo /odom</code>
Topic rate	<code>rostopic hz /scan</code>	<code>ros2 topic hz /scan</code>
List nodes	<code>rostopic list</code>	<code>ros2 node list</code>
Publish once (drive)	<code>rostopic pub /cmd_vel ...</code>	<code>ros2 topic pub /cmd_vel ...</code>
List services	<code>rosservice list</code>	<code>ros2 service list</code>
Params	<code>rosparam list</code>	<code>ros2 param list</code>
Visualize	<code>rviz</code>	<code>rviz2</code>
Node graph	<code>rqt_graph</code>	<code>rqt_graph (ros2 build)</code>
TF tree	<code>roslaunch rqt_tf_tree</code> <code>rqt_tf_tree</code>	<code>ros2 run rqt_tf_tree</code> <code>rqt_tf_tree</code>
Record a bag	<code>roslaunch record</code>	<code>ros2 bag record</code>

8. Tools that are ROS-version specific

ROS 1 only - `roscore`, `catkin_make` / `catkin build` - `move_base` navigation stack, `amcl` (ROS 1 build) - `map_server` (`map_saver`) - `robot_pose_ekf` (odometry fusion) - `ORB_SLAM2`, `rtabmap_ros` (visual SLAM — shipped in the ROS 1 Jetson branch) - `rviz` (v1)

ROS 2 only - `colcon` - `Nav2` (`navigation2`) with its behavior-tree navigator - `nav2_map_server` (`map_saver_cli`) - `robot_localization` (odometry fusion) - `slam_toolbox` / `slam_gmapping` (ROS 2 build) - `rviz2` - DDS middleware (no master; discovery is automatic)

Works in both (different builds — install the one matching your distro) - `roslaunch`, `rqt` tools, `gmapping`, `teleop_twist_keyboard`, YDLidar driver, `vision_opencv`

9. Common mistakes to avoid

1. **Don't mix workspaces.** Never source a ROS 1 `setup.bash` and a ROS 2 `local_setup.bash` in the same terminal — it corrupts the environment.
 2. **.launch vs .launch.py.** ROS 1 launch files end in `.launch` (XML); ROS 2 launch files end in `.launch.py` (Python). The file extension tells you which version it's for.
 3. **Package name typos.** `myagv_navigation` is ROS 1; `myagv_navigation2` is ROS 2. Same for `ydliar_ros_driver` (1) vs `ydliar_ros2_driver` (2).
 4. **roscore.** Required in ROS 1, does not exist in ROS 2. If a “cannot contact master” error appears, you are running a ROS 1 command on a ROS 2 system (or forgot `roscore`).
-

Appendix A — Rebuilding (only if the install is broken)

You normally never need this. The robot ships with ROS and the myAGV packages already installed and built. Use this section only if the workspace is damaged, files were deleted, or a package no longer launches and you need a clean rebuild. First confirm which version you are on (`printenv ROS_DISTRO`), then follow **only** that version's steps.

ROS 1 — rebuild

```
# Get the source (Jetson Nano branch)
git clone -b myagv_ros_2023JN https://github.com/elephantrobotics/myagv_ros.git
# (Raspberry Pi users: branch myagv_ros_2023Pi)
# Place package folders into ~/catkin_ws/src, then:

cd ~/catkin_ws
rosdep install --from-paths src --ignore-src -r -y
catkin_make
source ~/catkin_ws/devel/setup.bash      # add to ~/.bashrc for every terminal
```

ROS 2 — rebuild

```
# Get the source directly into a workspace src/ (Galactic, Jetson Nano branch)
git clone -b galactic-JN https://github.com/elephantrobotics/myagv_ros2.git myagv_ros2/src

cd ~/myagv_ros2
rosdep install --from-paths src --ignore-src -r -y

# extra apt deps required by Nav2 on Galactic:
sudo apt install ros-galactic-bondcpp \
  ros-galactic-test-msgs* \
  ros-galactic-behaviortree-cpp-v3* \
  ros-galactic-ompl \
  ros-galactic-joint-state-publisher \
  ros-galactic-rqt-tf-tree \
  ros-galactic-diagnostic-updater \
  ros-galactic-camera-info-manager -y

colcon build
source ~/myagv_ros2/install/local_setup.bash # add to ~/.bashrc for every terminal
```

Rebuild key differences - ROS 1 builds with `catkin_make`, ROS 2 builds with `colcon build`. - ROS 1 sources `devel/setup.bash`, ROS 2 sources `install/local_setup.bash`. - `catkin_make` in a ROS 2 workspace (or `colcon build` in a ROS 1 workspace) will fail.

Sources: elephantrobotics/myagv_ros (branch myagv_ros_2023JN, ROS 1) and elephantrobotics/myagv_ros2 (branch galactic-JN, ROS 2). Package lists and launch-file names verified against those repositories.